

Scripts d'évolution dans la démarche DevOps

Document dédié à la partie 1.3 Rédiger des scripts dans la démarche DevOps.

1.3.1 Les bases du déploiement automatique

Le déploiement automatique repose sur des scripts idempotents, exécutables en local comme en CI.

Implémentation réelle :

- `scripts/deploy-nas.sh` (build + vérifications + rsync optionnel)
 - `.github/workflows/cd-netlify.yml` (CD cloud)
 - `.github/workflows/cd-nas.yml` (CD NAS)
-

1.3.2 Rédiger et utiliser un script de déploiement

Le script `deploy-nas.sh` :

1. installe les dépendances,
2. exécute `lint` et `test:run`,
3. construit l'application,
4. synchronise les artefacts vers le NAS.

Commande :

```
./scripts/deploy-nas.sh  
NAS_HOST=... NAS_USER=... NAS_PATH=... ./scripts/deploy-nas.sh --rsync
```

1.3.3 Les bases des scripts d'évolution

Un script d'évolution de données doit :

- être **traçable** (logs clairs),
- intégrer des **contrôles de cohérence**,

- distinguer **dry-run** et **publication**.

Implémentation réelle :

- `scripts/run-evolution-pipeline.sh` (orchestrateur standard)
 - `scripts/convert_suivi_etpt_to_json.py` (conversion RenoiRH -> JSON)
-

1.3.4 Rédiger des scripts d'évolution

Le script `run-evolution-pipeline.sh` enchaîne :

1. conversion Excel -> JSON,
2. validation structurelle (`agents` non vide, IDs uniques, actifs présents),
3. empreinte SHA256 de la version générée,
4. push Neon optionnel (`--push-neon`).

Commandes :

```
bash scripts/run-evolution-pipeline.sh --file "trdata/Suivi_des_emplois_en_ETPT_RPROG (23).xlsx"
bash scripts/run-evolution-pipeline.sh --file "trdata/Suivi_des_emplois_en_ETPT_RPROG (23).xlsx" --push-neon
```

1.3.5 Optimiser les scripts d'évolution

Optimisations concrètes mises en place :

- mesure de durée par étape (timing ms),
- mode dry-run par défaut (réduit le risque),
- point de publication explicite (`--push-neon`),
- validations automatiques avant publication.

Ces optimisations réduisent les erreurs humaines et accélèrent le diagnostic.

1.3.6 Ecrire un script YAML d'Intégration Continue

Le workflow CI YAML (`.github/workflows/ci.yml`) automatise :

- lint,
- tests unitaires,

- audit npm,
 - build,
 - e2e.
-

1.3.7 Automatiser les tests en DevOps

Tests automatisés réellement actifs :

- `npm run test:run` (48 tests Vitest),
- `npm run test:e2e` (Playwright),
- `npm run security:test` (15 tests sécurité),
- `npm run audit:prod` (audit dépendances).

Exécution à chaque push via `ci.yml` .

Références

- `docs/COMPETENCE_DEPLOIEMENT_STUDI.md`
- `docs/DEVOPS.md`
- `docs/CD_PIPELINES.md`
- `scripts/README.md`